CertificateAuthority

System Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document describes implementation ownership, architecture, persistence, security and operational boundaries. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-CertificateAuthority-java-spring. 1 detailed Java SysDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-CertificateAuthority-go.

CertificateAuthority - Go 5.2.2

Standalone support system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/CertificateAuthority. Entry point: ./cmd/certificateauthority. Module: arrowhead/ah5-support-go/certificateauthority. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. The dedicated launcher selects only this support-system role. The formerly shared internal runtime and regression tests are copied locally, not linked from another tree. Files for common helpers and other route implementations remain in that private runtime; this packaging does not claim new functionality or Java parity. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8448 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8448"; 8540-8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
CertificateAuthority	Role/profile-constrained CSR signing, chain checks, certificate revocation, signed CRL export and trusted keys	Java SQL import, automatic CRL-file publication, wider relying-party revocation enforcement, HSM/OCSP/HA

Detailed implementation evidence

1 native Go SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-CertificateAuthority-rust.

CertificateAuthority (Rust)

Arrowhead 5.2.2 CertificateAuthority support system. Entry point src/main.rs is built as binary certificateauthority by the shared crate in ../ah5-support-runtime-rust (arrowhead-support-rust).

- examples/certificateauthority.json - example configuration (master copy: ../ah5-support-runtime-rust/examples/)

- documentation/ - SysD/SysDD/IDD/SD sources (symlink into ../ah5-support-runtime-rust/documentation/CertificateAuthority, where the contract generator writes)

Run: cargo run -p arrowhead-support-rust --bin certificateauthority -- <config.json>

Listening port

Default 127.0.0.1:8448 (Arrowhead default, role certificateauthority). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or --server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
CertificateAuthority	Native CSR verification/signing, profiles, revocation, PEM CRLs, trusted keys	Explicit SANs only (no CSR extension copying); automatic key renewal/root rollover, OCSP/HSM/HA, full validity/DTO normalization

Detailed implementation evidence

1 native Rust SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data

migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-CertificateAuthority-java-spring/documentation/v5.2.2/CertificateAuthority-v5.2.2-SysDD.tex

- Go: Eclipse-Arrowhead-go/ah5-CertificateAuthority-go/documentation/CertificateAuthority/CertificateAuthority-SysDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/CertificateAuthority/CertificateAuthority-SysDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

CertificateAuthority

System Design Description

Abstract
This document describes the design and implementation of the CertificateAuthority support system of Arrowhead 5.2.2 (`ah5-CertificateAuthority-java-spring`).

Contents

1 Overview **3**

2 Implementation **4**
2.1 Implementation language and tools 4
2.2 Functional properties implementation 4
2.3 Non functional properties implementation 5
2.4 Deployment 5
2.5 Local modifications 6
2.6 Verification 6

3 Services **7**

4 References **7**

5 Revision History **9**
5.1 Amendments 9
5.2 Quality Assurance 9

1 Overview

This document describes the CertificateAuthority system implementation, which realizes the CertificateAuthority SysD [1]. The CertificateAuthority is the local-cloud CA: it signs certificates under the Arrowhead profiles (onboarding, device, system, operator), validates certificates and trusted keys, keeps the certificate register and publishes the CRL. In Section 2, we describe implementation details of the system.

2 Implementation

This implementation realizes the CertificateAuthority SysD in `ah5-CertificateAuthority-java-spring`. Since 8 November 2026 it follows the Arrowhead Java-Spring pattern of the other support systems (for example Blacklist and TimeManager). The sources, a private copy of the Arrowhead common library (`vendor/`), tests, images and the deployment definition are owned by this folder. The earlier native implementation (JDK `HttpsServer`, SQLite) is no longer built; `legacy/` keeps the older history.

2.1 Implementation language and tools

- Java 21, Spring Boot 3.4, the Arrowhead common library (private source copy), Bouncy Castle for X.509, Hibernate/JPA with the MariaDB driver, Log4j2. Maven produces `arrowhead-certificate-authority-5.2.2.jar`; the entry point is `CertificateAuthorityMain`.
- Storage: a private MySQL schema `ah_certificate_authority` in the system's own database container.

2.2 Functional properties implementation

The following components implement the system:

Component	Responsibility
`CertificateAuthority SystemInfo`	System and service models (registered by the common `ApplicationInitListener` with `CERT_AUTH` interfaces).
`CertificateAuthorityAPI`, `GeneralManagementAPI`	HTTP operations; every path is served with and without the base path `/certificate-authority`.
`CertificateSigningService`	`sign`: CSR checks, caller profiles, reserved names, name ownership, validity limits, certificate issuing with `ca=false`.
`CertificateValidation Service`	`check-certificate`, `check-trusted-key`.
`CertificateManagement Service`	Certificate register, revocation (by id or content), trusted keys.
`CrlService`	CRL generation (`get-crl`), publication of `<crl.dir>/ca.crl` after each revocation and periodically.
`CallerProfileFilter`	Admits only system (`sy`) and operator (`op`) client certificates.
`CertificateAuthorityStatus Service`	`status` document and the sysMonitor echo detail.

Every request passes the common filter chain: the outsourced authentication filter verifies the identity token with Authentication and binds it to the client certificate; `CallerProfileFilter` checks the certificate profile; the management filter admits Sysop, CertificateAuthority and Onboarding to `/mgmt/` paths. The services then apply the CA's identity rules: Onboarding, Sysop and CertificateAuthority may sign (with profile limits), Sysop and CertificateAuthority manage, and Onboarding may revoke only the certificates it requested. The Blacklist filter is not used, so the CA keeps working while Blacklist is unavailable.

2.2.1 Data handled and stored

Schema `ah_certificate_authority`:

Tables	Content
`issued_certificate`	Certificate register: serial number, common name, requesting identity, validity, revocation time, PEM.
`trusted_key`	Trusted public keys with validity; a SHA-256 of the PEM keeps them unique.
`logs`	Operational log entries (`get-log`).

The management responses keep the field names of the native implementation (for example `serial_number`, `revoked_at`). `scripts/migrate-sqlite-to-mysql.py` imports a native SQLite store, keeping the ids.

2.3 Non functional properties implementation

2.3.1 Security

The HTTPS server requires TLS 1.2/1.3 with client certificates, checks them against the cloud CRL (`arrowhead.ssl.crl-file`) and accepts only certificate-bound identity tokens. The common `Security ProfileGuard` refuses to start with TLS disabled or declared identities unless a development profile is selected explicitly. The issuer certificate and signing key are PEM files mounted read-only; only this system mounts the key. The database account holds data-manipulation privileges on its own schema only. Errors use the Arrowhead `ErrorMessageDTO`.

2.3.2 Power management

None; the system is a server process without power states. The `sysMonitor power` observation always reports line supply.

2.3.3 Internal monitoring

Logs go to stdout, a rolling file and the `logs` table. The common `SysMonitorPublisher` publishes echo, power and status every 30 s with the routing key `<cloud>.sysmonitor.CertificateAuthority.certificateAuthority`; the echo detail carries the certificate counts.

2.3.4 Configuration

Configuration is `config/application.properties`, with environment placeholders for the cloud name, organization, registry, broker and CRL timing; secrets come from `/run/secrets` (`configtree`). The CA-specific keys are `ca.certificate.file`, `ca.key.file`, `crl.dir`, `crl.validity.hours`, `crl.refresh.seconds`, `cert.validity.days`, `onboarding.cert.validity.hours` and `reserved.names`.

2.4 Deployment

The local `Dockerfile` builds only this folder and runs `arrowhead-certificate-authority-standalone:5.2.2`. `compose.yaml` adds the private database `certificate-authority-db` (MySQL 8.4, TLS required, internal network, no host port). Its init script creates the schema and restricts the application account to `SELECT, INSERT, UPDATE, DELETE`. The HTTPS API uses port 8448. The application runs as UID 10001 with a read-only root filesystem; only the CRL output directory is writable. This packaging is supplied for operator deployment; existing test-cloud deployments are not changed by it.

ServiceRegistry is the bootstrap endpoint; Authentication and DynamicServiceOrchestration are looked up there. AMQPS monitoring is a network dependency. The CRL output directory must be distributed to the other systems of the cloud by the operator.

2.5 Local modifications

Notes on this implementation in the workspace:

- Native Java port (2026-10-16) of the Python CA; Java-Spring implementation (2026-11-08).
- Reserved names; LocalCloudManager no longer signs (J6); CRL publication (H3).
- Since the Spring implementation, every operation (including `check-certificate`, `get-crl` and `/health`) needs a certificate-bound identity token, and `generalManagement` adds `get-log` and `get-config`.

2.6 Verification

The JUnit tests cover the signing rules, revocation, CRL content and publication, certificate and trusted-key validation, the caller profile check, and every 5.2.2 route with and without the base path. An end-to-end test runs the whole application (filters, controllers, JPA) against a fresh MySQL schema created by the deployment init script with the restricted account. Startup with the development sample is refused by the security profile guard. A run in a live cloud with Onboarding is not part of this verification.

3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`certificateSigning`	[1]	[2]	[3]
`certificateValidation`	[1]	[4]	[5]
`certificateAuthority Management`	[1]	[6]	[7]
`sysMonitor`	[1]	[8]	[9]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
systemDiscovery, serviceDiscovery	[10]	[11]	–
identity	[12]	[13]	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

4 References

[1] J. Delsing, “CertificateAuthority – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-CertificateAuthority-java-spring/documentation/v5.2.2/CertificateAuthority-v5.2.2-SysD.pdf.

[2] ——, “certificateSigning – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.2.2, 2026, certificateAuthority-v5.2.2-certificateSigning-SD.pdf.

[3] ——, “certificateSigning HTTP/TLS/JSON – Interface Design Description (CertificateAuthority),” Eclipse Arrowhead, IDD 5.2.2, 2026, certificateAuthority-v5.2.2-certificateSigning-HTTP-IDD.pdf.

[4] ——, “certificateValidation – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.2.2, 2026, certificateAuthority-v5.2.2-certificateValidation-SD.pdf.

[5] ——, “certificateValidation HTTP/TLS/JSON – Interface Design Description (CertificateAuthority),” Eclipse Arrowhead, IDD 5.2.2, 2026, certificateAuthority-v5.2.2-certificateValidation-HTTP-IDD.pdf.

[6] ——, “certificateAuthorityManagement – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.2.2, 2026, certificateAuthority-v5.2.2-certificateAuthorityManagement-SD.pdf.

[7] ——, “certificateAuthorityManagement HTTP/TLS/JSON – Interface Design Description (CertificateAuthority),” Eclipse Arrowhead, IDD 5.2.2, 2026, certificateAuthority-v5.2.2-certificateAuthorityManagement-HTTP-IDD.pdf.

[8] ——, “sysMonitor – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.2.2, 2026, certificateAuthority-v5.2.2-sysMonitor-SD.pdf.

[9] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (CertificateAuthority),” Eclipse Arrowhead, IDD 5.2.2, 2026, certificateAuthority-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[10] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[11] ——, “systemDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-systemDiscovery-SD.pdf.

[12] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[13] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Independent Java build, private helpers and state, per-system deployment and configuration.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)
4	2026-11-08	5.2.2	Java-Spring implementation: common library, filter chain, JPA on a private MySQL schema, migration from SQLite.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 CertificateAuthority

1.1 System overview

Profile-constrained CSR issuance, revocation and trusted keys. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

1.2 Produced and consumed services

Produced HTTPS services: `certificateAuthorityManagement`, `certificateSigning`, `certificateValidation`, `echo`, `generalManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

1.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- CSR and CRL operations use a separately provisioned CA key; no CA key is generated/copied by this port. CRL export does not prove enforcement by every relying party. Java SQL import, automatic CRL file publication, HSM/OCSP/HA are absent.
- The source reference uses a shared non-Spring HTTP dispatcher. Its route coverage is not inferred from Go route presence; manual JavaSupportRuntime review is required.

1.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

2 Implementation

Go 1.26 or newer builds the native binary. State is serialized as private JSON documents; updates clone state, validate, persist and then publish. Server timeouts and bounded message sizes constrain resource use. Persistent outboxes are at-least-once and cannot provide distributed exactly-once delivery. Detailed source, configuration and test instructions are in the repository README and verification report.

2.1 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete

behavioral conformance.

Method	Path	Access
POST	/certificate-authority/sign	identity
POST	/certificate-authority/checkCertificate	identity
GET	/certificate-authority/crl	identity
GET	/certificate-authority/mgmt/certificates	management
DELETE	/certificate-authority/mgmt/certificates/{id}	identity
GET	/certificate-authority/mgmt/keys	management
DELETE	/certificate-authority/mgmt/keys/{id}	identity
POST	/certificate-authority/mgmt/revocations	management
PUT	/certificate-authority/mgmt/keys	management
POST	/certificate-authority/checkTrustedKey	identity
GET	/certificate-authority/monitor/echo	certificate
GET	/certificate-authority/general/mgmt/get-config	management
POST	/certificate-authority/general/mgmt/logs	management

1 CertificateAuthority

1.1 System overview

Native Rust development profile for local Java source `JavaSupportRuntime/`. One independently configured executable, identity, port and state file per role. No Java, Go or Python process implements the service. Some essential operations are explicitly unavailable.

1.2 Design and trust boundaries

Tokio/Hyper/Axum with rustls enforces mutual TLS. Certificate chain, validity, local CN/profile and startup-configured optional CRLs are checked. CRLs need process restart to refresh; without CRLs no revocation enforcement is claimed. Tokens are verified with Authentication and bound to the TLS identity. Business calls require ConsumerAuthorization; management requires a verified sysop or explicit broad whitelist. CA provisioning instead applies Java-style named certificate-principal rights. Outbound authorities are pinned to expected CN/profile before credentials are sent, in addition to chain, SAN and time validation.

1.3 Persistent state

Independent locked atomic JSON replaces SQL. Clone/validate/fsync/rename commits prevent partially published local transactions. Rename is the commit point; directory-sync failure is an uncertain durable commit, not a rollback. Owner-only keys/password files are separate. Network effects are not distributed transactions; interrupted starts and bridge operations require reconciliation. No automatic Java state import, clustering or deployment is delivered.

1.4 Implemented behavior

Native CSR verification/signing, profiles, revocation, PEM CRLs, trusted keys

1.5 Open release gates

Explicit SANs only (no CSR extension copying); CRL file publication/reload, OCSP/HSM/HA, full validity/DTO normalization. All systems remain development profiles, not certified interchangeable Java replacements. See `COMPATIBILITY.md` for detailed DTO fields and restrictions; the Java sources remain the authoritative contract.

1.6 Produced and consumed services

Advertised implemented HTTPS definitions: `certificateAuthorityManagement`, `certificateSigning`, `certificateValidation`, `echo`, `generalManagement`.

Authorities consumed as applicable: Authentication, ServiceRegistry, ConsumerAuthorization and explicitly configured pinned adapter/provider origins. Only implemented operations are registered. Service versions are 1.0.0 except Gatekeeper/GateTunnel 5.2.2. No AMQP sysMonitor.observe implementation is advertised.

1.7 HTTPS operations

JSON unless raw configuration bytes, PEM CRL or echo text. Bodies: 1 MiB; remote responses: 8 MiB. Header: `Bearer IDENTITY-TOKEN//token`. Unknown routes return 404, absent dependencies 503. Unported

operations return 501 after applicable access checks and are excluded from registration. Route presence is not semantic parity.

Method	Path	Access / status
GET	/certificate-authority/general/mgmt/get-config	management
POST	/certificate-authority/general/mgmt/logs	management
GET	/certificate-authority/monitor/echo	certificate
POST	/certificate-authority/sign	certificate
GET	/certificate-authority/crl	certificate
POST	/certificate-authority/checkCertificate	certificate
POST	/certificate-authority/checkTrustedKey	certificate
GET	/certificate-authority/mgmt/certificates	certificate
DELETE	/certificate-authority/mgmt/certificates/{id}	certificate
POST	/certificate-authority/mgmt/revocations	certificate
GET	/certificate-authority/mgmt/keys	certificate
PUT	/certificate-authority/mgmt/keys	certificate
DELETE	/certificate-authority/mgmt/keys/{id}	certificate

Error envelope: errorMessage, errorCode, exceptionType, origin and timestamp. Full Java status, DTO and validation equivalence requires differential testing.

2 References and qualification

See `contracts.json` for local Java SHA256 hashes/mappings, `COMPATIBILITY.md` for DTO details and gaps, and `VERIFICATION.md` for tests actually run. No upstream release-tag provenance, complete replacement or real-cloud interoperability is asserted.